Задача 3.1.6b

Условие: Отсортировать через QuickSort последовательность $a_1, \ldots, a_n$ в порядке возрастания элементов.

in: $n, a_1, \ldots, a_n$
out: $b_1, \ldots, b_n$, где $b_1 \leq \cdots \leq b_n$

Псевдокод

Источник: https://gitlab.com/freepascal.org/fpc/source/-/blob/a399f7898cd93a353973f1aa5aadd591ee83c76b/demo/text/qsort.pp#1

```
procedure sort(l, r: longint);
var
    i, j, x, y: longint;
begin
    i := l;
    j := r;
    x := a[(l+r) div 2];
    repeat
        while a[i] < x do
            inc(i);
        while x < a[j] do
            dec(j);
        if not (i > j) then
        begin
            y := a[i];
            a[i] := a[j];
            a[j] := y;
            inc(i);
            j := j - 1;
        end;
    until i > j;
    if l < j then
        sort(l, j);
    if i < r then
        sort(i, r);
end;

begin
    max := len(array);
    sort(1, max);
end;
```

Решение

```
Read 1
Load =0
Sub 1
JZero END
```

```
JGTZ END    ; n <= 0 ==> end
Load 1
Add =10
Store 2     ; r2 := 10 + n, end of array + 1

Load =10
Store 1     ; r1 := 10, start of array. And also r1 is a helping register

READ_ARRAY: Load 2
    Sub 1
    JZero INIT_SORT ; r1 == r2 => end read
    Read *1         ; buf[r1] <- In
    Load =1
    Add 1
    Store 1         ; r1 += 1
    Jump READ_ARRAY
INIT_SORT: Load =1
    Store 3         ; r3 := l = 1
    Load 2
    Store 4         ; r4 := &r = 10 + n
    Sub =10
    Store *4        ; r (= *r4) := n
    Load 4
    Add =2
    Store 9         ; r9 := &cs
    Load =0
    Store *9        ; cs := 0 (call from MAIN)
SORT: Load 4
    Add =1
    Store 5         ; r5 := &i = &r + 1 (next i on call stack)
    Load 3
    Store *5        ; i (= *r5) := l
    Load *4
    Store 6         ; r6 := j = last r on call stack
    Load 3
    Add *4
    Div =2          ; r0 := (l+r) // 2
    Add =9          ; r0 := &a[(l+r) // 2]
    Load *0
    Store 7         ; r7 := x = a[(l+r) // 2]
REPEAT: Jump WHILE_i
WHILE_i: Load *5    ; r0 := i
    Add =9          ; r0 := &a[i]
    Load *0         ; r0 := a[i]
    Sub 7
    JGTZ WHILE_j
    JZero WHILE_j   ; if a[i] - x >= 0 end while
    Load *5
    Add =1
    Store *5         ; i++
```

```
    Jump WHILE_i
WHILE_j: Load 6     ; r0 := j
    Add =9          ; r0 := &a[j]
    Load *0
    Sub 7
    Mult =-1         ; r0 := (a[j] - x) * (-1)
    JGTZ IF_not_i_gt_j
    JZero IF_not_i_gt_j ; if x - a[j] >= 0 end while
    Load 6
    Sub =1
    Store 6         ; j-
    Jump WHILE_j
IF_not_i_gt_j: Load *5
    Sub 6
    JGTZ UNTIL      ; if i > j do not do IF
    Load *5         ; r0 := i
    Add =9          ; r0 := &a[i]
    Load *0         ; r0 := a[i]
    Store 8         ; r8 := y = a[i]
    Load *5
    Add =9
    Store 1         ; r1 := &a[i]
    Load 6
    Add =9          ; r0 := &a[j]
    Load *0         ; r0 := a[j]
    Store *1        ; a[i] := a[j]
    Load 6
    Add =9
    Store 1         ; r1 := &a[j]
    Load 8
    Store *1        ; a[j] := y
    Load *5
    Add =1
    Store *5        ; i++
    Load 6
    Sub =1
    Store 6         ; j-
UNTIL: Load *5
    Sub 6
    JGTZ IF_l_lt_j  ; if i > j end repeat
    Jump REPEAT
IF_l_lt_j: Load 3
    Sub 6
    JGTZ IF_i_lt_r
    JZero IF_i_lt_r ; if l - j >= 0 do not do IF
    ; l = l
    Load 4
    Add =3
    Store 4          ; r4 := &r + 3 (next r)
    Load 6
```

```
    Store *4        ; r := j
    Load 9
    Add =3
    Store 9         ; &cs += 3
    Load =1
    Store *9        ; cs := 1 (call from sort(l, j))
    Jump SORT       ; call sort(l, j)
IF_i_lt_r: Load *5
    Sub *4
    JGTZ CALL_STACK_RETURN
    JZero CALL_STACK_RETURN    ; if i - r >= 0 do not do IF
    Load *4
    Store 1         ; r1 := r
    Load 4
    Add =3
    Store 4         ; &r += 3
    Load 1
    Store *4        ; r := r1 = r_previous
    Load *5
    Store 3         ; l := i
    Load 5
    Add =3
    Store 5         ; &i += 3
    Load 9
    Add =3
    Store 9         ; &cs += 3
    Load =2
    Store *9        ; cs := 2 (call from sort(i, r))
    Jump SORT       ; call sort(i, r)
CALL_STACK_RETURN: Load *9
    JZero PRINT_INIT    ; cs == 0 => exit sort(1, max)
    Load 4
    Sub =3
    Store 4         ; &r -= 3 (previous r)
    Load 5
    Sub =3
    Store 5         ; &i -= 3 (previous i)
    Load *9
    Store 1         ; r1 := cs
    Load 9
    Sub =3
    Store 9         ; &cs -= 3 (previous cs)
    Load 1
    Sub =1
    JZero IF_i_lt_r     ; cs == 1 => exit sort(l, j)
    Jump CALL_STACK_RETURN
PRINT_INIT: Load =10
    Store 1
PRINT: Load 2
    Sub 1
```

```
    JZero END       ; r2 == r1 => end print
    Write *1
    Load 1
    Add =1
    Store 1         ; r1 += 1
    Jump PRINT
END: Halt 0
```

Сложность

До вызова SORT:

1. Проверка n и инициализация - 10 шагов
2. Чтение массива READ_ARRAY - n итераций:
   (a) 3 шага на проверку конца массива
   (b) 5 шагов на тело цикла
3. Последняя проверка конца массива - 3 шага
4. Инициализация сортировки и стека INIT_SORT - 11 шагов

Значит до вызова SORT время - $8n + 24$ шагов.

После сортировки:

1. Подготовка PRINT_INIT - 2 шага
2. Вывод массива PRINT - n итераций:
   (a) 3 шага на проверку конца массива
   (b) 5 шагов на тело цикла
3. Последняя проверка конца массива - 3 шага
4. Завершение программы - 1 шаг

Итого, после сортировки время - $8n + 6$ шагов.

Значит, общее время вне SORT:

$$T_{outside}(n) = 16n + 30$$

Стоимость разбиения зависит от массива. В худшем суммарном случае опорный элемент минимальный. Рассмотрим плохое разбиение отрезка длины $m = r - l + 1 \geq 5$, при котором опорный элемент $x = a[p]$, $p = \lfloor (l + r)/2 \rfloor$ минимальный, и остаётся правая подзадача длины $m - 1$. Пусть u - количество итераций WHILE_j в первом проходе, v - количество итераций WHILE_j во втором проходе. Тогда

$$u = r - p, v = p - l - 1, u + v = m - 2$$

1. Инициализация SORT до REPEAT - 13 шагов
2. Первый проход через REPEAT, WHILE_i, WHILE_j:
   (a) Переход и проверка WHILE_i - $1 + 5$ шагов

(b) WHILE_j - u итераций по 11 шагов и завершающая проверка на опорном элементе - 7 шагов

(c) IF_not_i_gt_j - 3 шага, обмен и изменение i, j - 22 шага

(d) UNTIL - проверка и переход к следующему проходу - 4 шага

(e) Итого:

$$1 + 5 + 11u + 7 + 3 + 22 + 4 = 11u + 42$$

3. Второй проход через REPEAT, WHILE_i, WHILE_j:

(a) Переход и проверка WHILE_i - $1 + 5$ шагов

(b) WHILE_j - v итераций по 11 шагов и завершающая проверка - 7 шагов

(c) В IF_not_i_gt_j - 3 шага

(d) UNTIL - 3 шага

(e) IF_l_lt_j и IF_i_lt_r - по 4 шага

(f) Итого:

$$1 + 5 + 11v + 7 + 3 + 3 + 4 + 4 = 11v + 27$$

4. От IF_i_lt_r до Jump SORT - 18 шагов

Значит от входа в SORT до вызова правой подзадачи:

$$T_{sort}(m) = 13 + (11u + 42) + (11v + 27) + 18 = 13 + 11(m - 2) + 69 + 18 = 11m + 78$$

После завершения рекурсивного вызова $cs = 2$. CALL_STACK_RETURN - 17 шагов. Значит

$$T_{call}(m) = 11m + 78 + 17 = 11m + 95$$

Для отрезка длины 4 рассмотрим массив $[4, 3, 2, 1]$. Опорный элемент $x = a_2 = 3$. После двух обменов получаются подзадачи $[1, 2]$ и $[3, 4]$.

Сначала посчитаем затраты одной подзадачи длины 2:

1. Инициализация в SORT - 13 шагов

2. От REPEAT до UNTIL:

- REPEAT - 1 шаг
- проверка WHILE_i - 6 шагов
- в WHILE_j первая проверка - 7 шагов, уменьшение j - 4 шага, повторная проверка - 7 шагов, всего 18 шагов
- проверка IF_not_i_gt_j и обмен - $3 + 22$ шага
- завершающая проверка UNTIL - 3 шага

Итого разбиение: $1 + 6 + 18 + 3 + 22 + 3 = 53$ шага

3. Проверка IF_l_lt_j: переход выполняется после 3 шагов

4. Проверка IF_i_lt_r - 4 шага

Значит, одна подзадача длины 2 занимает

$$13 + 53 + 3 + 4 = 73$$

шага без учета возврата.

1. Инициализация в SORT - 13 шагов
2. Разбиение от REPEAT до выхода через UNTIL:
   - первый проход: REPEAT - 1 шаг, WHILE_i - 5 шагов WHILE_j - 6 шагов, проверка IF_not_i_gt_j и обмен - $3 + 22$ шага, UNTIL с переходом к повтору - 4 шага
   - второй проход: REPEAT - 1 шаг, WHILE_i - 6 шагов; WHILE_j - 6 шагов, проверка IF_not_i_gt_j и обмен - $3 + 22$ шага, UNTIL - 3 шага

   Итого разбиение:

$$1 + 5 + 6 + 3 + 22 + 4 + 1 + 6 + 6 + 3 + 22 + 3 = 82$$

3. Проверка IF_l_lt_j - 4 шага
4. Левый вызов: подготовка стека до Jump SORT - 11 шагов, SORT для подзадачи - 73 шага, возврат с $cs = 1$ в CALL_STACK_RETURN - 16 шагов
5. После возврата проверка IF_i_lt_r - 4 шага
6. Правый вызов: подготовка стека до Jump SORT - 18 шагов, SORT для подзадачи - 73 шага, возврат с $cs = 2$ в CALL_STACK_RETURN - 17 шагов

Финальная обработка возврата главного вызова - 2 шага. Значит:

$$T_{SORT,body}(4) = 13 + 82 + 4 + (11 + 73 + 16) + 4 + (18 + 73 + 17) + 2 = 313$$

Значит для $n \geq 4$:

$$T(n) = 311 + \sum_{m=5}^{n}(11m + 95) + 2 = \frac{11n^2 + 201n - 354}{2}$$

Для малых входов экспериментально программа даёт:

$$T(1) = 109, T(2) = 137, T(3) = 262$$

- тут уже с учетом outside.

И если добавить $T_{outside}(n) = 16n + 30$, то получаем:

$$T_{\max}(n) = \begin{cases} 109, & n = 1 \\ 137, & n = 2 \\ 262, & n = 3 \\ \dfrac{11n^2 + 233n - 294}{2}, & n \geq 4 \end{cases}$$